NAME:TANKISO MASOEBE

OPERATING SYSTEM (advanced)wk10

1.PERSONAL DIARY PROGRAM:

C++ Source Code

```
#include <iostream>
#include <fstream>
#include <string>
#include <ctime>
using namespace std;

int main() {
    int choice;
    string entry;

    do {
        cout << "\n===== PERSONAL DIARY =====\n";
        cout << "1. Add a new entry\n";
        cout << "2. Read all entries\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        cin.ignore();

        if (choice == 1) {
            ofstream file("diary.txt", ios::app);

            if (!file) {
                cout << "Error opening diary file!\n";
                continue;
            }

            time_t now = time(0);
            char* timestamp = ctime(&now);

            cout << "Write your diary entry: ";
            getline(cin, entry);

            file << "Date and Time: " << timestamp;
            file << "Entry: " << entry << "\n";
            file << "--------------------------\n";

            file.close();

            cout << "Diary entry saved successfully!\n";
        }
        else if (choice == 2) {
            ifstream file("diary.txt");
```

```
            if (!file) {
                cout << "No diary entries found yet.\n";
                continue;
            }

            cout << "\n===== YOUR DIARY ENTRIES =====\n";

            string line;
            while (getline(file, line)) {
                cout << line << endl;
            }

            file.close();
        }
        else if (choice == 3) {
            cout << "Exiting the diary. Goodbye!\n";
        }
        else {
            cout << "Invalid choice. Please try again.\n";
        }

    } while (choice != 3);

    return 0;
}
```

Explanation of the program

1. Add a new entry: The user types a diary entry, and the program saves it in diary.txt along with the current date and time.
2. Append mode: ios::app ensures that new entries are added to the end of the file without deleting previous entries.
3. Read previous entries: The program opens diary.txt and displays every saved entry using a while loop.
4. Timestamp: time(0) gets the current time, while ctime() converts it into a readable date and time.
5. Menu system: A do-while loop keeps displaying the menu until the user chooses to exit.
6. File handling: ofstream writes to the file, while ifstream reads from it.

Sample output

```
===== PERSONAL DIARY =====
1. Add a new entry
2. Read all entries
3. Exit
Enter your choice: 1
```

```
Write your diary entry: Today I attended my programming class.
Diary entry saved successfully!

===== PERSONAL DIARY =====
1. Add a new entry
2. Read all entries
3. Exit
Enter your choice: 2

===== YOUR DIARY ENTRIES =====
Date and Time: Wed Oct 14 18:00:00 2026
Entry: Today I attended my programming class.
-------------------------
```

How to run it in Code::Blocks

1. Open Code::Blocks.
2. Create a new Console Application and select C++.
3. Replace the contents of main.cpp with the code above.
4. Click Build and Run (F9).
5. Add an entry, then select option 2 to read it.

2. Here is a C++ CSV File Parser Program suitable for a college assignment. It reads data from data.csv, separates each line into fields, displays the information in a formatted way, and writes the processed data to a new file called formatted_data.txt.

1. C++ Source Code

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

int main() {
    ifstream inputFile("data.csv");
    ofstream outputFile("formatted_data.txt");

    string line;
    string name, age, profession;

    if (!inputFile.is_open()) {
        cout << "Error: Could not open data.csv" << endl;
        return 1;
    }
```

```
    if (!outputFile.is_open()) {
        cout << "Error: Could not create output file." << endl;
        return 1;
    }

    cout << "===== EMPLOYEE DATA =====" << endl;

    cout << "Name\t\tAge\tProfession" << endl;
    cout << "----------------------------------------" << endl;

    outputFile << "EMPLOYEE INFORMATION\n";
    outputFile << "====================\n\n";

    while (getline(inputFile, line)) {
        stringstream ss(line);

        getline(ss, name, ',');
        getline(ss, age, ',');
        getline(ss, profession, ',');

        cout << name << "\t\t"
             << age << "\t"
             << profession << endl;

        outputFile << "Name: " << name << endl;
        outputFile << "Age: " << age << endl;
        outputFile << "Profession: " << profession << endl;
        outputFile << "--------------------" << endl;
    }

    inputFile.close();
    outputFile.close();

    cout << "\nData processed successfully!" << endl;
    cout << "Formatted data saved to formatted_data.txt" << endl;

    return 0;
}
```

2. Input file: **data.csv**

Create a file named data.csv in your program's working directory and add the following information:

```
John,25,Engineer
Jane,30,Designer
Peter,28,Teacher
Mary,24,Nurse
```

3. Sample console output

```
===== EMPLOYEE DATA =====
Name            Age     Profession
----------------------------------------
John            25      Engineer
Jane            30      Designer
Peter           28      Teacher
Mary            24      Nurse

Data processed successfully!
Formatted data saved to formatted_data.txt
```

4. Output file: **formatted_data.txt**

The program creates a new file containing the processed information in a different format.

```
EMPLOYEE INFORMATION
====================

Name: John
Age: 25
Profession: Engineer
--------------------
Name: Jane
Age: 30
Profession: Designer
--------------------
Name: Peter
Age: 28
Profession: Teacher
--------------------
Name: Mary
Age: 24
Profession: Nurse
--------------------
```

5. Explanation of the program

1. File reading: ifstream opens data.csv to read the employee records.
2. File writing: ofstream creates formatted_data.txt to store the processed information.
3. Reading each line: getline(inputFile, line) reads one record at a time.
4. Parsing CSV data: stringstream and getline(ss, name, ',') separate the name, age, and profession using commas as delimiters.
5. Formatted display: cout displays the extracted fields in columns.
6. Writing processed data: outputFile writes the same information using labels instead of comma-separated values.
7. Closing files: close() closes both files after processing.

6. How to run it in Code::Blocks

1. Create a new C++ Console Application.
2. Paste the source code into main.cpp.
3. Create data.csv in the program's working directory.
4. Enter the sample records shown above.
5. Click Build and Run (F9).
6. Check the console output and open formatted_data.txt to see the reformatted data.

Conclusion: This program demonstrates file handling, string parsing, loops, and formatted output in C++. It converts comma-separated employee records into readable console output and a structured text file.

3.OBJECT PERCIISTENCE:

Object persistence means saving objects to a file so that their data remains available after the program closes.

1. C++ Source Code

This example uses file handling and a simple text-based serialization technique. Each book is written to the file as three separate lines.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
using namespace std;

class Book {
public:
    string title;
    string author;
    int year;

    Book() {
        year = 0;
    }

    Book(string t, string a, int y) {
        title = t;
        author = a;
        year = y;
    }

    void display() const {
```

```
        cout << "Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Year: " << year << endl;
        cout << "-----------------------" << endl;
    }
};

void saveBooks(const vector<Book>& books) {
    ofstream file("books.txt");

    if (!file) {
        cout << "Error saving books!" << endl;
        return;
    }

    for (const Book& book : books) {
        file << book.title << endl;
        file << book.author << endl;
        file << book.year << endl;
    }

    file.close();
    cout << "Books saved successfully!" << endl;
}

void loadBooks(vector<Book>& books) {
    ifstream file("books.txt");

    if (!file) {
        cout << "No saved collection found. Starting a new one."
             << endl;
        return;
    }

    string title, author;
    int year;

    while (getline(file, title) &&
           getline(file, author) &&
           (file >> year)) {

        file.ignore(10000, '\n');
        books.push_back(Book(title, author, year));
    }

    file.close();
    cout << "Saved books loaded successfully!" << endl;
}

int main() {
    vector<Book> books;
    int choice;
```

```
// Load previously saved books when the program starts
loadBooks(books);

do {
    cout << "\n===== BOOK COLLECTION =====" << endl;
    cout << "1. Add a new book" << endl;
    cout << "2. Display all books" << endl;
    cout << "3. Save collection" << endl;
    cout << "4. Exit" << endl;
    cout << "Enter your choice: ";
    cin >> choice;
    cin.ignore(10000, '\n');

    if (choice == 1) {
        string title, author;
        int year;

        cout << "Enter book title: ";
        getline(cin, title);

        cout << "Enter author: ";
        getline(cin, author);

        cout << "Enter publication year: ";
        cin >> year;
        cin.ignore(10000, '\n');

        books.push_back(Book(title, author, year));

        cout << "Book added successfully!" << endl;
        saveBooks(books);
    }
    else if (choice == 2) {
        if (books.empty()) {
            cout << "No books in the collection." << endl;
        } else {
            cout << "\n===== SAVED BOOKS =====" << endl;

            for (const Book& book : books) {
                book.display();
            }
        }
    }
    else if (choice == 3) {
        saveBooks(books);
    }
    else if (choice == 4) {
        saveBooks(books);
        cout << "Exiting program. Goodbye!" << endl;
    }
    else {
        cout << "Invalid choice. Try again." << endl;
    }
```

```
    } while (choice != 4);

    return 0;
}
```

2. Sample output

```
Saved books loaded successfully!

===== BOOK COLLECTION =====
1. Add a new book
2. Display all books
3. Save collection
4. Exit
Enter your choice: 1

Enter book title: Things Fall Apart
Enter author: Chinua Achebe
Enter publication year: 1958

Book added successfully!
Books saved successfully!
```

If the user selects option 2, the program displays:

```
===== SAVED BOOKS =====
Title: Things Fall Apart
Author: Chinua Achebe
Year: 1958
------------------------
```

3. Explanation of the program

1. Book class: Defines the attributes title, author, and year, which store information about each book.
2. Vector: The vector<Book> collection stores multiple book objects in memory.
3. Serialization: The saveBooks() function converts each book object's attributes into text and writes them to books.txt.
4. Deserialization: The loadBooks() function reads the saved information and reconstructs the book objects when the program starts.
5. Adding books: The user enters the book details, and the program adds the new object to the collection.
6. Displaying books: The display() function prints the details of each book.
7. Persistence: The collection remains saved in books.txt, so it can be loaded the next time the program runs.

4. Conclusion

The program demonstrates object persistence in C++ using a Book class, a vector collection, and file handling. Serialization saves object data to a file, while deserialization restores that data when the program starts. This allows the book collection to be maintained between program